Pérez Aguiar Oropeza Gabriel Emiliano
UNAM / IIMAS / LCD
Semestre 2026-2
Reconocimiento de Patrones

Práctica 1: Preprocesamiento de datos y extracción de características

Objetivos

- Identificar y analizar la presencia de correlaciones positivas entre las variables de un conjunto de datos.
- Aprender a extraer características de imágenes utilizando técnicas de procesamiento de imágenes.
- Observar de manera experimental los efectos del procesamiento de datos, imágenes y señales.
- Familiarizarse con el proceso de limpieza y preprocesamiento de los datos.
- Aprender a generar señales sintéticas y visualizarlas para su análisis o procesamiento posterior.

Introducción

Preprocesamiento y limpieza de Datos para el Reconocimiento de Patrones

El preprocesamiento de datos es una etapa fundamental en el reconocimiento de patrones, ya que garantiza la calidad y relevancia de los datos antes de su análisis. Este proceso incluye varias técnicas como limpieza, transformación, reducción y normalización de datos.

La limpieza de datos implica la eliminación de valores atípicos, datos incompletos o inconsistentes, mientras que la transformación convierte los datos en un formato adecuado, como la codificación de variables categóricas o la aplicación de transformaciones matemáticas. La normalización y estandarización aseguran que los datos tengan una escala uniforme, lo cual es crucial para algoritmos que dependen de medidas de distancia. Además, en el caso de datos textuales o de imágenes, se aplican técnicas específicas como la 'tokenización' o el filtrado de ruido.

Un buen preprocesamiento no solo mejora la precisión de los modelos de reconocimiento de patrones, sino que también reduce el tiempo de procesamiento y evita problemas de sobreajuste.

En este proceso se aplican diferentes técnicas de análisis de datos que van desde la visión por computadora hasta el procesamiento de señales biomédicas. Las principales técnicas utilizadas en el preprocesamiento de datos son:

- **Limpieza de Datos:** Consiste en eliminar errores, valores atípicos y datos faltantes para asegurar la calidad de los datos antes de su uso en modelos de reconocimiento de patrones. Las principales funciones son: el manejo de valores faltantes, eliminación de outliers y corrección de inconsistencias.

- **Normalización y Estandarización:** Este conjunto de funciones aseguran que las características tengan escalas comparables, mejorando la estabilidad y rendimiento de los algoritmos de reconocimiento de patrones. Entre los principales métodos se encuentran: la normalización (Min-Max Scaling), estandarización y escalamiento.

- **Transformaciones de Datos:** Mejora la representación de los datos para aumentar la capacidad predictiva y mejorar la separación entre clases. Estas transformaciones pueden ir desde una raíz cuadrada, hasta transformadas de Fourier y Wavelet. Así mismo, se suele utilizar una codificación 'One-Hot' que convierte variables categóricas en vectores binarios, y la codificación de frecuencia, que asigna valores numéricos en función de la frecuencia de aparición de una categoría.

- **Filtrado y Eliminación de Ruido:** Se aplica principalmente en señales e imágenes para mejorar la calidad de los datos y facilitar su análisis. Algunos de los filtros que se utilizan son: filtros de suavizado, filtro Notch, filtros pasa-bajo y pasa-alto.

- **Integración y Formateo de Datos:** Cuando se utilizan múltiples fuentes de datos, es esencial integrarlas y formatearlas de manera coherente. Algunas de las técnicas que podemos utilizar son: la conversión de formatos; que transforma los tipos de datos para su compatibilidad con diferentes herramientas de análisis, y la reducción de redundancia; que utilizamos para la eliminación de información duplicada para evitar sesgos en el análisis.

ECG (electrocardiograma)

La señal electrocardiográfica (ECG) es un registro de la actividad eléctrica del corazón y se utiliza ampliamente en el diagnóstico de enfermedades cardiovasculares. El ECG está compuesto por diversas ondas (ver fig. 1), incluyendo la onda P, el complejo QRS y la onda T, que reflejan la despolarización y repolarización del corazón. Existen diversos métodos para el análisis de una señal ECG, como la descomposición de Fourier; que permite separar sus componentes y mejorar la interpretación de los datos.

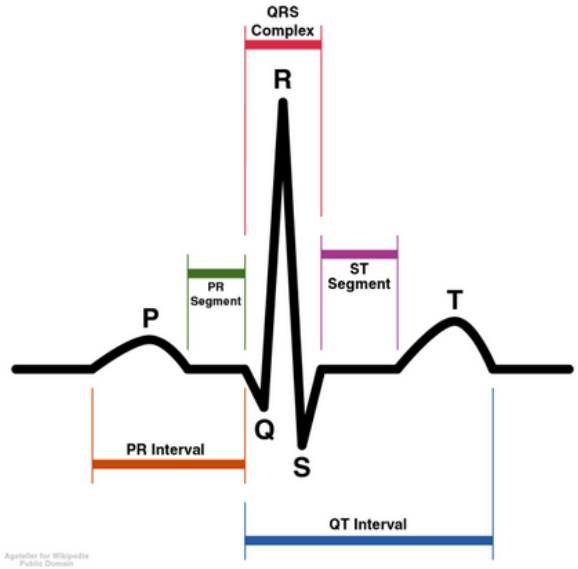


Fig. 1 Representación esquemática de un ECG normal (Anthony Atkielski).

Las señales ECG pueden presentar contaminación por diversas fuentes de ruido, tales como la interferencia de línea de alimentación y los artefactos de movimiento. Por consiguiente, resulta indispensable el uso de técnicas de filtrado para mejorar la calidad del análisis. En este contexto, el desarrollo de algoritmos de clasificación ha permitido mejorar la detección de patologías cardíacas a partir de estas señales.

Desarrollo

Ejercicio 1: Análisis de correlación en el conjunto de datos de la flor Iris

1. Carga del conjunto de datos de la flor Iris.

   Se importó la biblioteca `pandas` con el fin de mejorar el manejo de datos tabulares, y la función `load_iris` del módulo `sklearn.datasets`. Se definió el objeto `data` para almacenar el conjunto de datos de la flor Iris. Posteriormente, se asignó la variable `df = data.frame` para extraer el 'DataFrame', el cual es un objeto de `pandas` que contiene los datos y variables a utilizar. Finalmente, se mostró una vista previa de los primeros cinco registros del conjunto de datos.

2. Selección de las 4 variables numéricas.

   Se generó un nuevo dataframe a partir de `df`, el cual contiene únicamente las variables numéricas de la flor: `sepal length (cm)`, `sepal width (cm)`, `petal length (cm)` y `petal width (cm)`. Después, se mostró una vista previa de los primeros 3 registros del nuevo objeto.

3. Cálculo de la matriz de correlación entre estas variables.

   Utilizando las variables del ejercicio anterior, se generó la matriz de correlación utilizando el método `.corr()` de `pandas`, con el parámetro `method='pearson'` para calcular la correlación utilizando el coeficiente de Pearson. Posteriormente, se imprimió la matriz para su visualización.

4. Identificación de correlaciones positivas altas.

   Se identificaron tres pares de variables con una relación positiva significativa: la longitud del sépalo y del pétalo, que presentan una correlación de 0.87; la longitud y anchura del pétalo, que muestran una relación fuerte con un coeficiente de 0.96; y, por último, la longitud del sépalo y la anchura del pétalo, con una correlación de 0.81.

5. Visualización de correlaciones.

   Se elaboró un mapa de calor (heatmap) de correlaciones entre las 4 variables de la flor Iris. Se utilizó la biblioteca `seaborn` para la creación del mapa de calor, y el módulo `matplotlib.pyplot` para la generación de un título y una visualización más agradable. A continuación se muestra la figura obtenida (ver fig. 2).

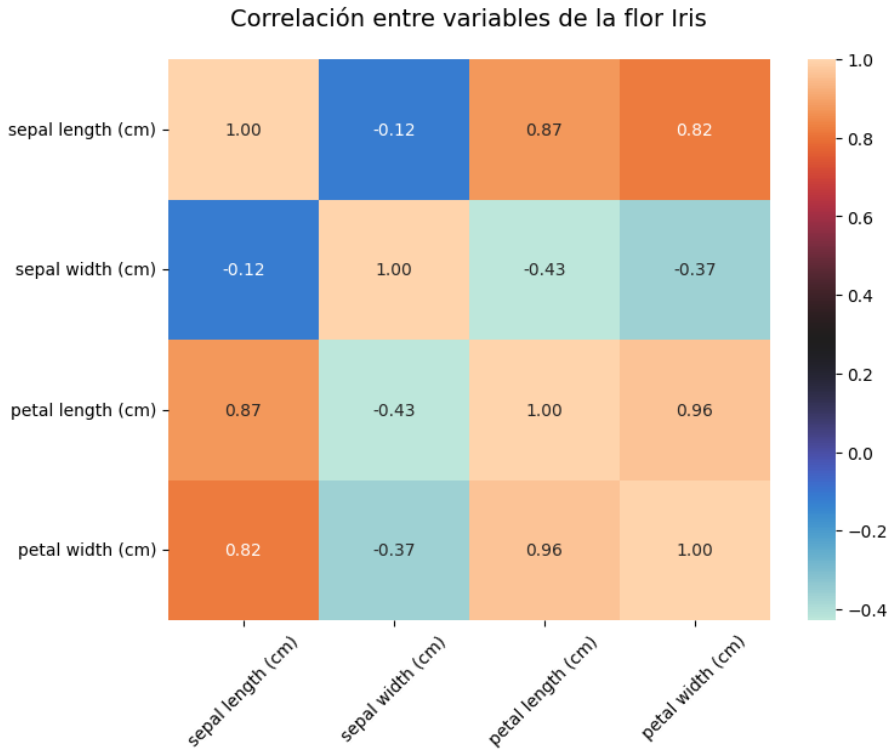


Fig. 2 Mapa de calor de correlaciones entre variables de Iris.

6. Interpretación de resultados.

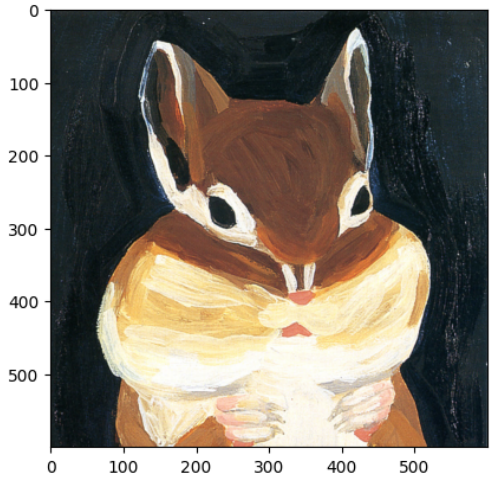
   Tras el análisis de la matriz de correlación, se identificaron relaciones significativas entre algunas de las características físicas de la flor Iris. Se distinguen tres pares de variables con alta correlación:

   - Longitud y anchura del pétalo (0.96): Una correlación casi de 1 indica que el pétalo tiende a crecer de forma proporcional en ambas dimensiones.
   - Longitud del pétalo y sépalo (0.87): Nos dice que existe una relación fuerte entre la longitud de ambas estructuras.
   - Longitud del sépalo y anchura del pétalo (0.81): Indica que estas características presentan una tendencia de crecimiento muy similar.

Ejercicio 2: Extracción de características en imágenes

1. Carga de una imagen.

   Se utilizó la biblioteca OpenCV para la lectura de una imagen de 600x600 pixeles. Debido a que OpenCV utiliza por defecto el formato de color BGR, se realizó una inversión en el orden de los canales (la tercera dimensión del arreglo [:, :, ::-1]). Esto permite convertir la imagen al formato RGB, garantizando una visualización correcta de la imagen. Para la visualización de los resultados, se empleó la biblioteca Matplotlib.pyplot.



Carga de una imagen con OpenCV, visualizada en Matplotlib

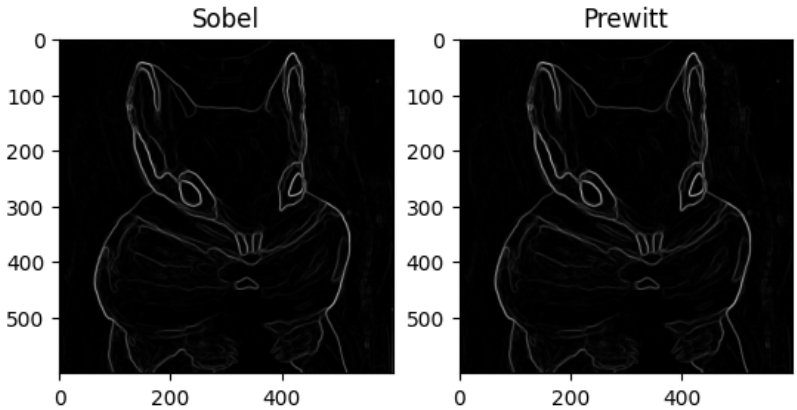
2. Preprocesamiento de la imagen.

   Se convirtió la imagen a escala de grises con el método `.cvtColor()` y el parámetro `COLOR_RGB2GRAY`. Además, se comprobó que, en efecto, la imagen pasó a tener una sola dimensión en lugar de 3 (las capas de color RGB). Finalmente, se decidió aplicar un filtro de suavizado 'medianBlur' para eliminar las texturas de pincelado de la imagen. Los resultados se muestran a continuación.

Imagen preprocesada después de la aplicación de filtros

3. Extracción de bordes.

   Tras comparar los resultados de los operadores 'Sobel' y 'Prewitt' para la extracción de bordes, se optó por el primero debido a que la imagen resultante presentó una mayor nitidez y una menor cantidad de ruido. Previo a esta operación, la imagen se convirtió a tipo float, esto con el motivo de vincular este proceso con la biblioteca scikit-image, la cual ofrecía mejores herramientas para esta tarea en particular. A continuación se muestran los resultados de ambos operadores.

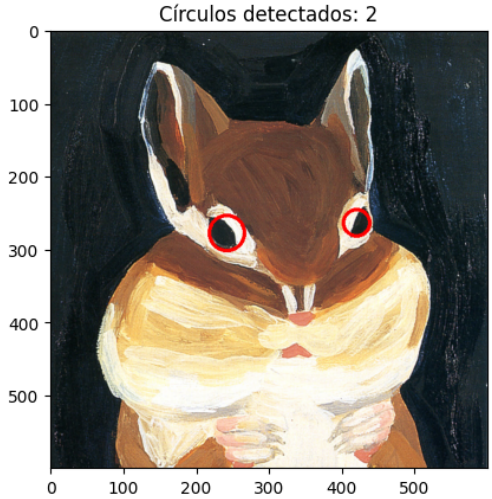


Operadores para la extracción de bordes

4. Descriptores de forma.

   Se utilizó la transformada de Hough para detectar círculos en la imagen, esperando identificar los ojos en la figura elegida. Previo a este proceso, la imagen se convirtió a tipo uint8 (que toma valores de 0 a 255), pues OpenCV utiliza esta lógica para el procesamiento de imágenes.

   Después de probar distintos parámetros para la función `.HoughCircles()`, se detectaron los ojos de la ardilla, es decir, los círculos que se querían encontrar. Finalmente, se realizó un plot de matplotlib con la imagen original y los círculos encontrados para visualizar los resultados obtenidos.



Detección de círculos con `.HoughCircles()`

   Por medio del módulo 'measure' de scikit-image, se detectaron regiones y sus propiedades utilizando region props. En particular, la función `regionprops` encontró 2 superficies mayoritarias: una con área de 215,368 y perímetro de 83,177, y otra con área de 55 y un perímetro de 50. Las demás regiones resultaron demasiado pequeñas en comparación, con apenas un área de 1 unidad o menores.

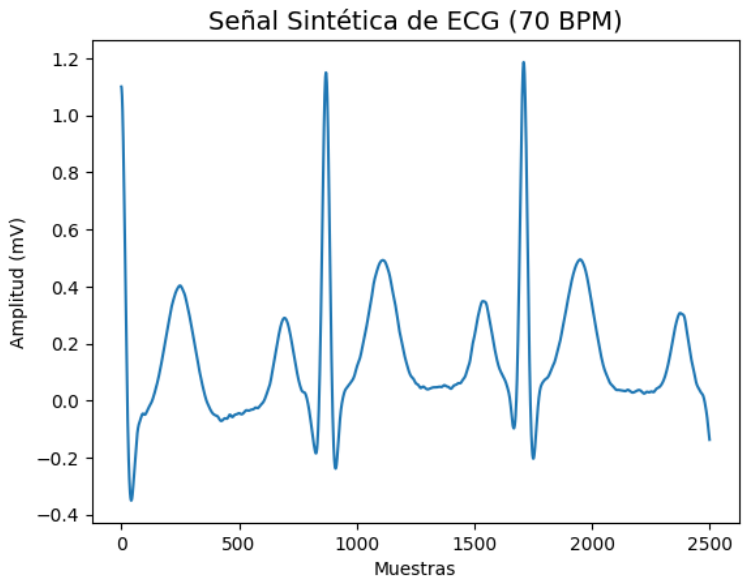
5. Comparación de resultados.

   Durante la etapa de preprocesamiento, se determinó que la conversión a escala de grises y la aplicación de un filtro de suavizado fueron pasos esenciales para realizar la detección de bordes. Por otro lado, aunque fue necesario ajustar los parámetros de la transformada de Hough para detectar únicamente los círculos de interés, el algoritmo detectó estas formas en la mayoría de los casos. Por último, si bien el análisis de region props identificó 1,250 regiones, se concluyó que solo existen dos regiones significativas: la figura principal y el fondo.

Ejercicio 3: Generación de señales sintécticas de ECG

1. Generación de una señal sintética de ECG.

   Por medio de la función `ecg_simulate()` de la biblioteca Neurokit2 de python se generó una señal sintética de ECG. Esta función incluye las ondas P, QRS y T de una onda ECG. Se ajustaron los parámetros para que la señal tuviera una duración de 10 segundos, un ritmo cardíaco de 70 bpm (beats per minute), y una frecuencia de muestreo de 1,000 hz. Finalmente, se ploteó una muestra de la señal.



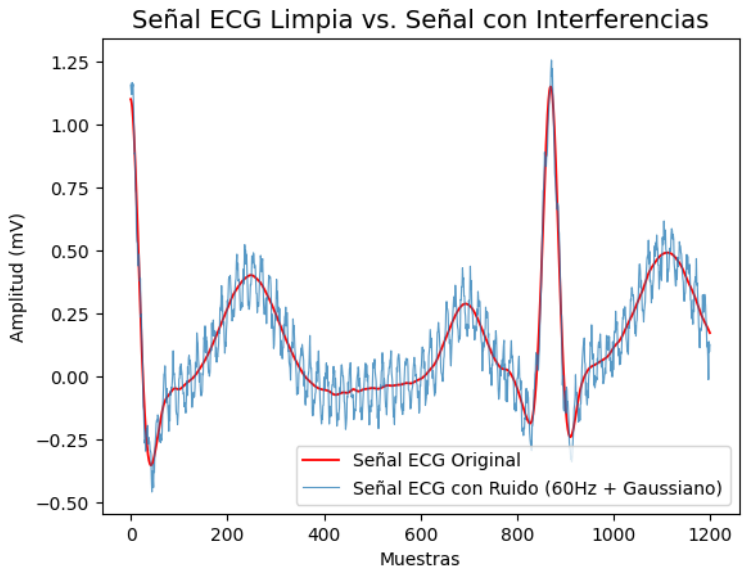
Señal de ECG generada con `ecg_simulate()`

2. Simulación de ruido en la señal.

   Se simularon dos tipos de ruido: una señal de ruido de alta frecuencia y ruido aleatorio. Posteriormente, se añadió el ruido a la señal original y se realizó una visualización comparativa entre ambas señales.

   El ruido de alta frecuencia fue generado con la función `.signal_simulate()` de Neurokit2, con la misma duración y frecuencia de muestreo que la señal original, con una frecuencia de 60 hz y una amplitud de las oscilaciones de 0.1.

   Por otro lado, el ruido aleatorio fue generado con el módulo random de la biblioteca Numpy. Esta onda esta dada por una distribución normal con media 0 y varianza 0.03, con la misma longitud de la señal original, tratando de emular una señal de ruido blanco gaussiano.

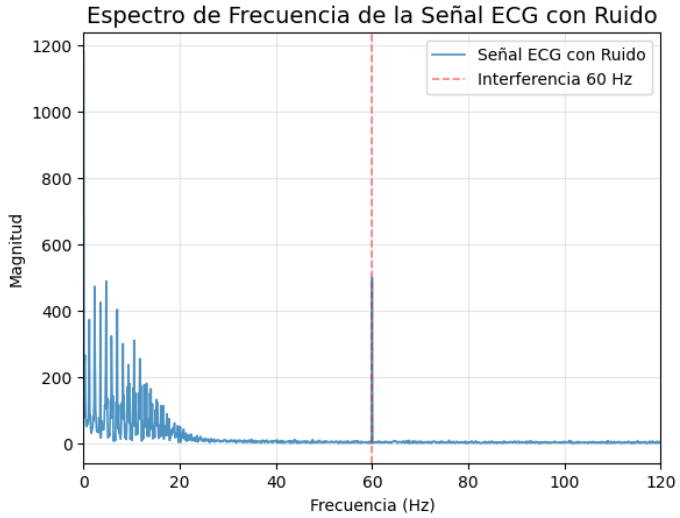


Señal original vs señal ECG con ruido

3. Análisis del espectro de frecuencia con la transformada de Fourier (FFT).

   Se generaron las frecuencias de muestreo discretas que corresponden a los componentes de la Transformada Rápida de Fourier Real (RFFT) utilizando la función `fft.rfftfreq()` de numpy. Posteriormente, se calculó la Transformada Rápida de Fourier Discreta (FFT) con la función `np.fft.rfft()`. Sin embargo, como la salida de esta función es un arreglo complejo, se calculó el valor absoluto de cada elemento, lo cual devuelve las magnitudes de cada componente de frecuencia de la señal.

   Para analizar el espectro de frecuencia, se realizó una gráfica comparativa entre las frecuencias (eje x) y las magnitudes (eje y). Es fácil observar que en la frecuencia de 60 hz existe un pico o interferencia, generada por el ruido de alta frecuencia del ejercicio anterior.



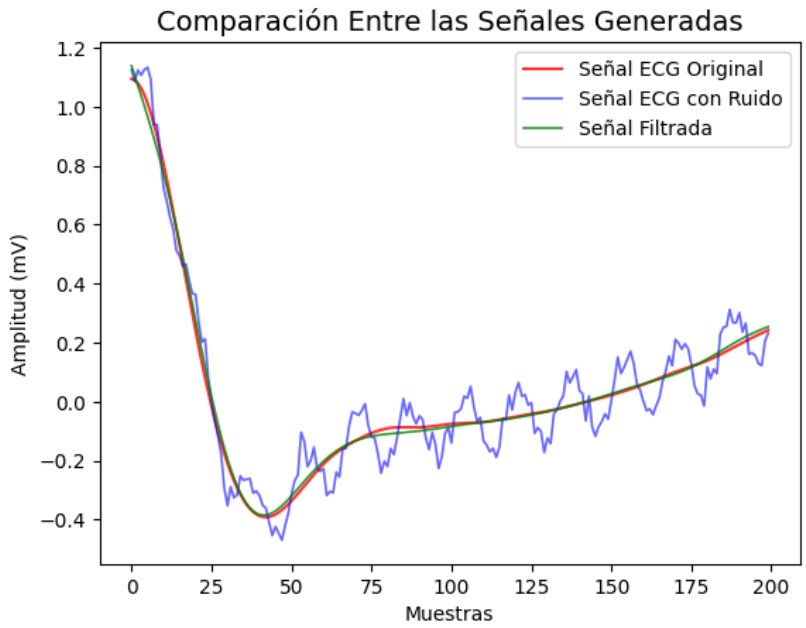
Visualización del espectro de frecuencias dado por la Transformada Rápida de Fourier

4. Creación de un filtro para eliminar el ruido de alta frecuencia.

   Se implementó un filtro con una frecuencia de corte de 40 hz para eliminar el ruido de frecuencias por encima de este umbral. Tras aplicar el filtro a la señal con ruido, esta se mostró sumamente similar a la señal ECG original, con pequeñas diferencias únicamente perceptibles al ampliar el rango de frecuencia de muestreo.

5. Comparación de señales y análisis de resultados obtenidos.

   Al comparar a las tres señales en una gráfica, podemos observar la efectividad del filtro utilizado, pues logró atenuar significativamente las oscilaciones generadas por el ruido gaussiano y el ruido de 60 hz, logrando un resultado casi perfecto.



Comparación de señales para observar el efecto del filtrado

Código

Ejercicio 1

```
# Bibliotecas necesarias para este ejercicio
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

# 1.1. Carga del conjunto de datos de la flor Iris
data = load_iris(as_frame=True)
df = data.frame

display(df.head(5))

# 1.2 Selección de las 4 variables numéricas
variables = df[['sepal length (cm)', 'sepal width (cm)',
                'petal length (cm)', 'petal width (cm)']]

display(variables.head(3))

# 1.3 Cálculo de la matriz de correlación entre estas variables
corr_matrix = (variables).corr(method='pearson')

print(corr_matrix)

# 1.5 Visualización de las correlaciones con un mapa de calor
plt.figure(figsize=(8,6))
sns.heatmap(corr_matrix, annot=True, cmap='icefire' , fmt=".2f")
plt.title('Correlación entre variables de la flor Iris', fontsize=14, pad=20)
plt.xticks(rotation=45)
plt.show()
```

Ejercicio 2

```
# Bibliotecas necesarias para este ejercicio
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import filters, util
from skimage.measure import label, regionprops

# 2.1. Carga de la imagen
img = cv2.imread('polaris.jpeg')[:, :, ::-1]

print('Tamaño de la imagen: ', img.shape)
plt.imshow(img)
plt.show()
```

```python
# 2.2. Preprocesamiento de la imagen

# Conversión a escala de grises
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
print('Tamaño de la imagen (ya no debe tener 3 dimensiones): ', gray.shape)

# Aplicación de filtro de suavizado (medianBlur)
smoothed = cv2.medianBlur(gray, 7)

# Visualización
fig, axes = plt.subplots(1, 2)
axes[0].imshow(gray, cmap='gray')
axes[0].set_title('Escala de grises')
axes[1].imshow(smoothed, cmap='gray')
axes[1].set_title('Suavizado (medianBlur)')
plt.show()

# 2.3. Extracción de bordes

# Conversión de imagen a float (0 - 1.0) para procesamiento con scikit-image
smoothed_sk = util.img_as_float(smoothed)

# Aplicación de filtros (sobel y prewitt)
sobel = filters.sobel(smoothed_sk)
prewitt = filters.prewitt(util.img_as_float(smoothed_sk))

# Visualización
fig, axes = plt.subplots(1, 2)
axes[0].imshow(sobel, cmap='gray')
axes[0].set_title('Sobel')
axes[1].imshow(prewitt, cmap='gray')
axes[1].set_title('Prewitt')
plt.show()

# 2.4. Descriptores de forma

# Conversión de imagen a uint8 (0 - 255) para procesamiento con OpenCV
edges = (sobel * 255).astype(np.uint8)

# Detección de círculos
circles = cv2.HoughCircles(edges, cv2.HOUGH_GRADIENT, dp=1,
                           minDist=40, param1=50, param2=25,
                           minRadius=15, maxRadius=30)

# Visualización de los círculos detectados
display = img.copy()
circles = np.round(circles[0, :]).astype("int")
```

```
for (x, y, r) in circles:
    cv2.circle(display, (x, y), r, (255, 0, 0), 3)

plt.imshow(display)
plt.title(f'Círculos detectados: {len(circles)}')
plt.show()

# Extracción de propiedades de regiones con region props

# Antes de usar regionprops, se debe etiquetar la imagen
label_img = label(edges > 0)

# Extracción de propiedades
props = regionprops(label_img)

# Propiedades de las primeras 3 regiones
print(f"Se encontraron {len(props)} regiones.\n")
row, col = region.centroid
for i, region in enumerate(props[:3]):
    print(f"Región {i+1}:")
    print(f"  Área: {float(region.area):.2f}")
    print(f"  Perímetro: {region.perimeter:.2f}")
    print("-" * 40)
```

Ejercicio 3

```
# Bibliotecas necesarias para este ejercicio
import numpy as np
import neurokit2 as nk
import matplotlib.pyplot as plt

# 1. Generación de una señal sintética de ECG

# ecg_simulate genera automáticamente las ondas P, QRS y T
ecg = nk.ecg_simulate(duration=10, sampling_rate=1000, heart_rate=70)

# Visualización de la señal
plt.plot(ecg[:2500])
plt.title('Señal Sintética de ECG (70 BPM)', fontsize=14)
plt.xlabel('Muestras')
plt.ylabel('Amplitud (mV)')
plt.show()

# 2. Simulación de ruido en la señal

# Ruido de alta frecuencia (60 Hz)
_60hz = nk.signal_simulate(duration=10, sampling_rate=1000,
                           frequency=60, amplitude=0.1)
```

```python
# Ruido aleatorio (ruido blanco gaussiano)
white_noise = np.random.normal(0, 0.03, len(ecg))

# Combinación de la señal más el ruido
ecg_final = ecg + _60hz + white_noise

# Visualización de la nueva señal
plt.plot(ecg[:1200], label='Señal ECG Original', color='red', lw=1.3)
plt.plot(ecg_final[:1200], label='Señal ECG con Ruido (60Hz + Gaussiano)', lw=0.8, alpha=0.75)
plt.title('Señal ECG Limpia vs. Señal con Interferencias', fontsize=14)
plt.legend()
plt.xlabel('Muestras')
plt.ylabel('Amplitud (mV)')
plt.show()

# 3. Transformada de Fourier para analizar el espectro de frecuencia
freqs = np.fft.rfftfreq(len(ecg), d=1/1000)
fft = np.abs(np.fft.rfft(ecg_final))

# Visualización del Espectro de Frecuencia
plt.plot(freqs, fft, lw=1.2, label='Señal ECG con Ruido', alpha=0.8)
plt.axvline(x=60, color='red', linestyle='--', alpha=0.5, lw=1.2, label='Interferencia 60 Hz')
plt.title('Espectro de Frecuencia de la Señal ECG con Ruido', fontsize=14)
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('Magnitud')
plt.xlim(0, 120)
plt.legend(loc='best')
plt.grid(True, alpha=0.3)
plt.show()

# 4. Creación de un filtro para eliminar el ruido de alta frecuencia
ecg_filter = nk.signal_filter(ecg_final, sampling_rate=1000, highcut=30,
                              method='butterworth', order=5)

# Visualización
plt.plot(ecg[:200], label='Señal ECG Original', color='red', lw=1.5, alpha=0.8)
plt.plot(ecg_final[:200], color='blue', label='Señal ECG con Ruido', lw=1.2, alpha=0.6)
plt.plot(ecg_filter[:200], color='green', label='Señal Filtrada', lw=1.2, alpha=0.8)
plt.title('Comparación Entre las Señales Generadas', fontsize=14)
plt.xlabel('Muestras')
plt.ylabel('Amplitud (mV)')
plt.legend()
plt.show()
```

Conclusiones

En esta práctica se exploraron las distintas etapas del procesamiento de datos de diversa índole, abarcando desde datos tabulares hasta imágenes y señales electrocardiográficas. Este proceso es fundamental para la creación y diseño de sistemas de reconocimiento de patrones.

- El primer ejercicio demostró la utilidad de un análisis de correlación en datos numéricos, pues resulta indispensable interpretar el grado de dependencia entre variables. En un sistema de reconocimiento de patrones, esta herramienta nos permite identificar las características que pueden ser redundantes para clasificar objetos en un modelo.
- El segundo ejercicio evidenció la importancia del preprocesamiento en la extracción de características en imágenes, así como la necesidad de seleccionar el filtro adecuado para la detección de bordes, círculos y líneas. Un proceso de extracción de características en imágenes debe ser capaz de identificar distinguir patrones para su clasificación precisa.
- Por último, el tercer ejercicio demostró la manera de generar, modificar y restaurar señales electrocardiográficas, así como la utilidad del análisis espectral. Este proceso resultó clave para identificar el impacto de frecuencias, ruido e interferencias sobre una señal biológica ECG.

Referencias

[1] Gonzalez, R., Woods, R., *Digital Image Processing*, Prentice Hall, 2008.

[2] Pratt, W. K., *Digital Image Processing*, John Wiley and Sons Inc, 2001.

[3] Jahne, B., *Digital Image Processing*, Springer, 2005.

[4] Sánchez Garreta, Josep Salvador; García, Vicente. Special Issue on *Data Preprocessing in Pattern Recognition: Recent Progress, Trends and Applications*. 2022.

[5] Majidi, Mehrdad; Oskuoee, Mohammad. Improving pattern recognition accuracy of partial discharges by new data preprocessing methods. *Electric Power Systems Research*, 2015, vol. 119, p. 100-110.

[6] Lotfi, Safia, et al. Recent progress on the synthesis, morphology and photocatalytic dye degradation of $BiVO_4$ photocatalysts: A review. *Catalysis Reviews*, 2024, vol. 66, no 1, p. 214-258.